

Congratulations on making it to the end in Java. For those where this was their first course in programming, nicely done getting to the end. This will likely be the hardest programming course you will take as you had to learn both programming and syntax. In future languages, knowing the specific syntax and nuances of other languages will be the hardest.

In this lab, you will be creating a license registration tracking system for the Country of Warner Brothers for the State of Looney Tunes. You will create four classes: Citizen, CarOwner, RegistrationMethods, and RegistrationDemo.

The UMLs for Citizen and CarOwner will be key to completing the four java files mentioned above.

Citizen
- firstName: String - lastName: String
+setFirstName(String inFirst): void +getFirstName(): String +setLastName(String inLast): void +getLastName(): String +toString(): String

Citizen Class

- It will also be necessary to create two constructors: (1) no arg constructor (2) constructor that accepts two String params to update firstName and lastName instance field vars. Create applicable javadocs including @param tags for the constructors.
- .toString() should return firstName, a space, and lastName.

CarOwner
- license: String - month: int - year: int
+setLicense(String inLicense): void +getLicense(): String +setmonth(int inMonth): void +getMonth(): int +setYear(int inYear): void +getYear(): int +toString(): String

CarOwner Class

- It will also be necessary to create two constructors: (1) no arg constructor (2) constructor that accepts three String params and two integers. You need to update firstName and lastName instance field vars. HINT – Use super() along with any arguments to update first and last. Set instance field vars license, month, and year. Create applicable javadocs including @param tags for the constructors.
- .toString() overrides and returns firstName, a space, and lastName, license, month/year. HINT – using super.toString() along with “\t” and “/” will help. **NOTE – Sometimes it is necessary to use**

two tabs to ensure that you go over enough space before writing the next item to line everything up. Pitfall – Sometimes the text output looks misaligned. If you print to pdf, you will see that the columns do line up.

RegistrationMethods

This is where most of the logic of the DMV is happening and where most of the coding effort is at for the Final Project. Key is to keep this modular. One method at a time, do not get overwhelmed. Recommend completing the methods in the order of what is required in RegDemo. This is also the order of methods in the provided RegistrationMethodsInterface.

- 1) `setFileNames( )`
 - a. Message the user to provide the path to the registration.csv file.
 - b. Use Scanner to create an object and use an applicable method to collect user input
 - c. Use a while pit to ensure that you have a valid file path to set the applicable instance field var. Since working with Scanner with a File object argument, you will have a checked exception and either need to throws IOException (or optionally set up a try and catch blocks).
 - d. Message the user to enter a path for the output.txt file, collect the path, and set the applicable instance field var.
- 2) `processTextToArray (CarOwner[ ] inArray )`
 - a. Set up a File object tied to inputFileName and pass to a Scanner constructor to set up a stream. You will need to do this in a try/catch.
 - b. Use `nextLine()` to get to the 2nd line since the first line is the column headers.
 - c. Use a while loop to test for `hasNextLine( )`. While true
 - i. Get the line
 - ii. Use `.split(???)` to tokenize. You have to replace the ??? with the correct item.
HINT-How are items in a csv file separated across a line
 - iii. Assign the token pieces to the array values as applicable.
 - iv. Create a CarOwner object and then add to the CarOwner[] passed into this method
 - v. You will need to create an int var (ie element set to 0) before the while loop. Each time you fill an element in the array, you will need to increment element to file the next CarOwner object into the next element.
 - d. Make sure to close out the stream
- 3) `printArrayToFile(CarOwner[ ] inArray, String inMsg)`
 - a. Set up a FileWriter object tied to outFileName with true to append and pass to a PrintWriter constructor to set up a stream. Since working with PrintWriter with a FileWriter object argument, you will have a checked exception and either need to throws IOException (or optionally set up a try and catch blocks).
 - b. Print inMsg to the stream
 - c. Run through the array either using a for or for each loop to print each item in the array to the output file stream
 - d. You will want to print an additional line in the file for spacing
 - e. Make sure to close out the stream
- 4) `flagOverdueOwners(CarOwner[] inArray)` - This will be similar to the `findAll( )` from Ch7

- a. Create an int with the monthsTotal. Remember each year has 12 months, so if you take REG_YEAR*12 and add REG_MONTH, you have the total months.
 - b. Create a counter set to 0
 - c. Make one pass using a for loop through inArray to see how many owners have registration that is over 12 months old. You can do this by subtracting the inArray[i].getYear()*12+inArray[i].getMonth() from totalMonths. If >12, increment the counter.
 - d. Create a new CarOwner[] sized to the counter
 - e. Make another pass using a for loop like in step c, but this time, assign the value to index 0, make sure to have an index var that is incremented each time you assign a value.
- 5) flagAlmostDueOwners(CarOwner[] inArray) – This is like flagOverdueOwners above
- a. Follow the steps for flagOverdueOwners above except in step c, the criteria will be
monthsTotal - (inArray[i].getYear()*12+inArray[i].getMonth())>9 &&
monthsTotal - (inArray[i].getYear()*12+inArray[i].getMonth())<=12

RegDemo

- 1) Create a RegistrationMethods object called `dmv`
- 2) Invoke `setFileNames( )` using the object created above
- 3) Invoke `getArraySize( )` and assign to a previously created int var (ie size).
- 4) Create a CarOwner[] called `ltState` set to size from above step.
- 5) Invoke `processTextToArray( )` passing in `ltState` as an argument
- 6) Invoke `printArrayToFile( )` passing the appropriate arguments (see RegistrationMethods methods summary above). For the String argument pass the following phrase "List of Car Owners"
- 7) Use `flagOverdueOwners( )` to create a new CarOwner[] whose registration is over 1 yr old that is assigned to CarOwner[] `overdue` based on passing in `ltState`.
- 8) Use the appropriate RegistrationMethods method to print `overdue` with the String header "Owners with Expired Registration"
- 9) Use `flagAlmostDueOwners( )` to create a new CarOwner[] whose registration will expire within the next 3 months or less, but is not over one year old (registrations that are 10-12 months old) that is assigned to CarOwner[] `almostDue` based on the array `ltState`.
- 10) Use the appropriate RegistrationMethods method to print `almostDue` with the String header "Owners with registration expiring in three months or less"
- 11) Use `getOutputFileName()` to get the path of the output and then print out to the screen where the output file is located.